

Deep-Learning for Volcanic Seismic Events Classification

Aaron Salazar, Rodrigo Arroyo, Noel Pérez and Diego Benítez

Colegio de Ciencias e Ingenierías “El Politécnico”,

Universidad San Francisco de Quito USFQ, Campus Cumbayá, Casilla Postal 17-1200-841, Quito, Ecuador
{assalazar, rodrigo.arroyo}@estud.usfq.edu.ec, {nperez, dbenitez}@usfq.edu.ec

Abstract—In this work, we proposed a new method to classify long-period and volcano-tectonic spectrogram images using six different deep learning architectures. The developed method used three deep convolutional neural networks named: DCNN1, DCNN2, and DCNN3. Also, three deep convolutional neural networks combined with deep recurrent neural networks named DCNN-RNN1, DCNN-RNN2, and DCNN-RNN3 to maximize the area under the curve of the receiver operating characteristic scores on a dataset of volcano seismic spectrogram images. The three deep recurrent neural network-based models reached the worst results due to the overfitting produced by the small number of samples in the training sets. The DCNN1 overcame the remaining models by touching an area under the curve of the receiver operating characteristic and accuracy scores of 0.98 and 95%, respectively. Although these values were not the highest values per metric, they did not represent statistical differences against other results obtained by more algorithmically complex models. The proposed DCNN1 model showed similar or superior performance when compared to the majority of the state of the art methods in terms of the accuracy metric. Therefore it can be considered a successful scheme to classify LP and VT seismic events based on their spectrogram images.

Keywords—volcanic seismic event classification, deep-learning models, artificial intelligence, spectrogram images.

I. INTRODUCTION

Volcanic monitoring systems are essential to detect early signs of volcanic unrest and possible reawakening that can lead to eruptions [1]. Amongst the techniques used by scientists to estimate activity inside a volcano, the seismicity is one of the most effective tools for monitoring and forecasting eruptions [2]. In this regard, a wide variety of approaches have been used in recent years to address the problem of volcano seismic events classification, e.g., long-period (LP) and volcano-tectonic (VT) seismic events, as it is shown in Fig. 1. Machine learning classifiers (MLC) such as hidden Markov models (HMM) [3], boosting strategies [4], decision trees (DT) [5], random forest (RF) [6], [7], Gaussian mixture models (GMM) [8], support vector machine (SVM) methods [7], [9], and artificial neural networks (ANN) [10], [11] were combined with classical time, frequency and scale domain features and non traditional as intensity statistic, shape and texture features extracted from the spectrogram images [7] to differentiate seismic events.

Work funded by Universidad San Francisco de Quito (USFQ) through the Poli-Grants Program under Grants 10100, 12494 and 16916.

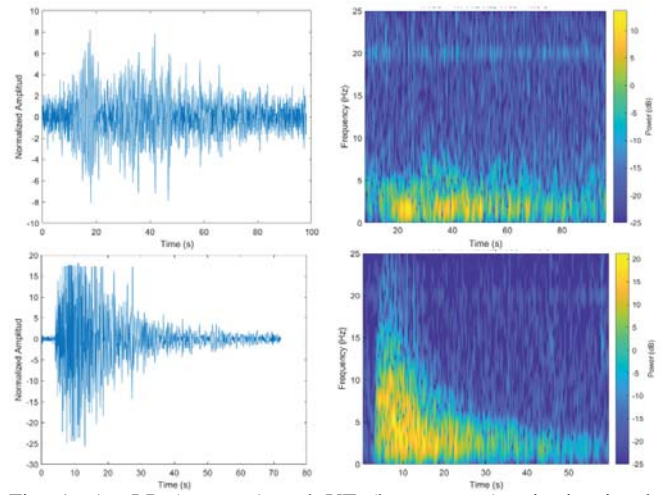


Fig. 1: An LP (top row) and VT (bottom row) seismic signals examples and their respective spectrogram. Taken from [12].

On the other hand, convolutional neural networks (CNN) are particular ANN architectures that are gaining more attention in image analysis contexts [13]. They avoid using intermediate, fully connected layers to employ pooling ones and thus optimizing the information pass-through from layer to layer. Lately, there is evidence of using deep learning techniques to analyze the seismic activity of volcanoes, e.g., deep neural networks to classify feature vectors computed from the time-domain signals [14], deep CNN models to classify spectrogram images [15]. Another well known deep learning architecture is the one based on recurrent neural network (RNN) where information flows sequentially, it is shared between layers and kept as a factor for decision making during the weight calculations [16]. The combination of both the CNN and RNN approach is quite possible, as it was demonstrated in [17] to classify different objects on individual images. However, the model growing is a must concerns aspect. The more internal layers are included in the desired model, the more complex it will be [18].

There are other deep learning-based approaches for object detection [16], [17] and classification [19]. But, the use of deep learning techniques in the context of volcano seismic event classification based on their spectrogram images is still limited. Therefore, in this work, we explore the use of six different deep learning architectures to classify LP and VT spectrogram

images. The developed method used single and combined deep CNN and RNN architectures to maximize the area under the curve (AUC) of the receiver operating characteristics curve on a dataset of volcano seismic spectrogram images from the Cotopaxi volcano, in Ecuador.

II. MATERIALS AND METHODS

A. Spectrogram images dataset

This work considered the use of a public dataset (*MicSigVI*) from the ESeismic repository, which contains several seismic event samples recorded at the Cotopaxi volcano [12]; available at http://www.igepn.edu.ec/eseismic_web_site/index.php. It has a total of 1187 seismic records from two different seismic stations (VC1 and BREF) installed at the Cotopaxi volcano. This dataset contains samples distributed in five classes: LP, VT, regional (REG), hybrid (HB), and icequakes (ICE). Due to the small number of samples from REG, HB, and ICE events, we considered only the LP and VT events belonging to the same seismic station (BREF) to guarantee the same acquisition protocol and to avoid mixed signals. Therefore, the formed experimental dataset contains 668 spectrogram images (587 of LP and 81 of VT).

B. Deep-learning networks

Deep learning can enhance computational models by including multiple layers to process large amounts of data and to improve the learning process. Thus, severe problems regarding image classification and recognition in the past are presently easier to tackle. The deep CNN and RNN are two exclusive deep learning models [16], [17], which are increasing their popularity on sequential data analysis and image labeling, respectively.

The deep CNN is a multilayered approach of conventional convolutional neural networks that include an input layer, a set of hidden layers (which could vary depending on the network architecture from two to hundreds of layers), and an output layer (fully connected layer). In deep CNN learning, each hidden layer is mainly composed of the CNN architecture core, consisting of at least the convolutional and max-pooling layers. Other configurations extend the basic scheme by adding dropout and flatten layers. This multi-layer structure enables the network to learn different data abstractions while transitioning from layer to layer until reaching the output result [15].

The deep learning RNN is based on the classic feed-forward ANN architecture, but it includes an extra working piece called loops in connections. In contrast to the feed-forward ANN, the RNN architecture processes the inputs in a sequential way considering a recurrent hidden state in which the current activation is dependant on the previous step activation. The main drawback is related to long-term sequential data, where the gradients tend to vanish during the training. However, there is a more sophisticated approach to design recurrent units and to avoid vanishing problems known as long short-term memory [20]. It allows for recurrent units to learn long-term

dependencies, which are a vital key when developing deep RNN models [21].

C. Proposed method

We adopted both the deep CNN and RNN neural networks to build the proposed method, which extends these neural networks to six different deep learning architectures: DCNN1, DCNN2, DCNN3, DCNN-RNN1, DCNN-RNN2, and DCNN-RNN3. For a better explanation of the proposed method, we focus our description in the DCNN1 and DCNN-RNN1 models. DCNN1 architecture is composed of several layers, as it is shown in Fig. 2.

From this figure, it is possible to read that the spectrogram images are used to feed the first convolutional layer composed of 16 convolutional filters with a 3×3 kernel size each. This layer aims to predict the class probabilities of the input sample by creating a feature map representation computed by the structure of the filters. Subsequently, the feature map enters the pooling layer with a 4×4 kernel size each to reduce irrelevant features (information) while retaining the relevant ones. Then, the reduced feature space is used to feed another convolutional and pooling layer with the same configurations as the previous ones. This second convolutional module concentrated the most relevant (important) features to classify the input sample. Finally, the fully connected layer consists of two dense layers, the flatten to convert the reduced bi-dimensional input feature space into a single feature vector with its corresponding weights and the output layer, which provides the final classification of the feature vector using a sigmoid function.

On the other hand, DCNN-RNN1 architecture is a mixed model that combines a two convolutional layer based deep CNN architecture with some extra RNN layers, as it is shown in Fig. 3. In such a sense, the first convolutional layer (Conv1.) used 32 convolutional filters with a 2×2 kernel size each and a pooling layer (Max_pool1) with a 3×3 kernel size. The second convolutional unit (Conv2. and Max_pool2 layers) used the same number of filters as in the Conv1., but it increased the convolutional kernel size to 5×5 . The Max_pool2 layer remained as equal as the Max_pool1 in terms of configurations. Then, a flatten and dense layer transforms the bidimensional feature space into a single feature vector that is the input to a repeat vector layer. The later transforms the input feature vector into a data stream that is propagated with a repeat factor of 30 as input to the first long-short term memory (LSTM) unit. The LSTM1 unit is composed of 1024 recurrent units (RU) with input shape (IS) size of 30 and input units (IU) number of 32. Then, the LSTM1 layer output feeds the LSTM2 unit that is set to 512 RU and the same input shape size and number of input units as the LSTM1 layer. After that, a fully connected layer containing a flatten and dense layers with sigmoid function provide the final classification. The remaining architectures, DCNN2, DCNN3, DCNN-RNN2, and DCNN-RNN3, follow the same base architecture (described here), varying the layers

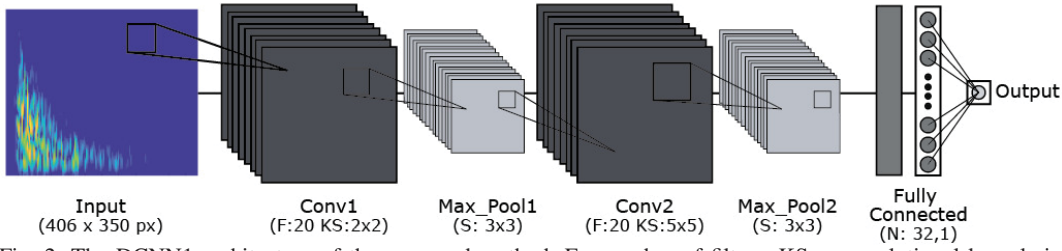


Fig. 2: The DCNN1 architecture of the proposed method; F - number of filters; KS - convolutional kernel size; S - max pool kernel size; N - number of neurons.

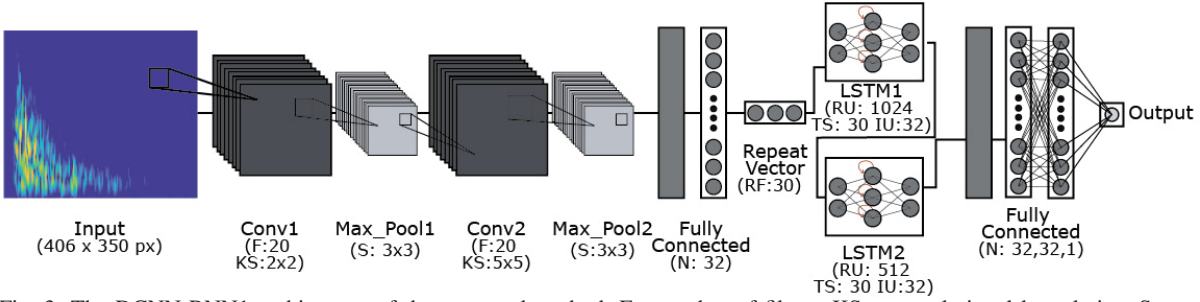


Fig. 3: The DCNN-RNN1 architecture of the proposed method; F - number of filters; KS - convolutional kernel size; S - max pool kernel size; N - number of neurons; RF - repeat factor; RU - recurrent units; TS - time stamp; IU - input units.

configurations and hyperparameters. The other architectures are summarized next:

The DCNN2 model contains three convolutional layers with 32, 64, and 128 filters with a kernel size of 3×3 each. Three max-pooling layers (one by each convolutional layer) with a pool size of 6×6 each, one flatten layer and a fully connected layer (output) composed of three dense layers (32, 32 and 1 neurons). The DCNN3 model uses two convolutional layers with 20 filters each, and a kernel size of 2×2 and 3×3 , respectively. Two max-pooling layers (one by each convolutional layer) with a pool size of 3×3 , one flatten layer, and a fully connected layer (output), containing three dense layers (32, 32, and 1 neurons).

The DCNN-RNN2 model employs three convolutional layers with 20 filters each and kernel size of 2×2 , 2×2 , and 5×5 , respectively. Three max-pooling layers (one by each convolutional layer) with a pool size of 3×3 , one flatten layer, one dense layer with 32 neurons, one repeat vector layer with a repetition factor of 30 units, two LSTM layers: the first one with 1024 recurrent units and input shape 30×32 , and the second one with 512 recurrent units and a fully connected layer (output), composed of three dense layers (32, 32 and 1 neurons).

Finally, the DCNN-RNN3 model involves three convolutional layers with 32 filters each and with a kernel size of 2×2 each. Three max-pooling layers (one by each convolutional layer) with a pool size of 3×3 , one flatten layer, one dense layer with 32 neurons, one repeat vector layer with a repetition factor of 30 units, two LSTM layers with 512 and 256 recurrent units, an input shape of 30×32 , and a fully connected layer (output), containing three dense layers (32, 32 and 1 neurons).

D. Experimental setup

1) *Spectrogram image preprocessing*: all spectrogram images were downscaled to 50% from their original size, thus decreasing the volume of information used to feed the learning models. This dataset provides spectrogram images without noise; therefore, the seismic event pattern presented on each image is invariant to the downscaling operation. This operation is use frequently in image analysis context with deep learning [15]. Besides, the pixels values of each spectrogram image were normalized using the min-max method [22] to bring them into the range of 0 to 1, thus, avoiding data dispersion. Besides, we used a data augmentation technique to increase and balance the number of samples per class. Thus, each spectrogram image underwent shearing, scaling, and rotation operations, as defined in [23]. Affinity transformations are widely used [15] and allowed us to reach a total of 1108 spectrogram images, which reinforces the models learning process by training them with more samples per class, helping to avoid overfitting.

2) *Training and test partitions*: the stratified 10-fold cross-validation method [24] was applied before the classification step to build disjoint training and test partitions and to ensure the sample ratio between both types of events for all folds. Thus, individual deep learning models were trained using different training sets, which enable it to learn from different input space representations. Testing on these different sets promotes trustworthy resulting variability in the classification of individual samples.

3) *Deep architectures configurations*: for all models, we configured three main hyperparameters to explore the proposed method limits. Thus, the number of iterations (epochs) was set from 50 to 150 with increment step of 50 units; the batch size was tuned to 16, 32 and 64 units, and the learning rate used the *adam* optimizer, which is based on adaptive estimation

TABLE I: Performance results of deep learning Models selected by the first selection criterion.

Architecture	Conv. layer (f)	Kernel size	Pool size per layer	FC layer (n)	Batch size	Epochs (u)	AUC	Wilcoxon at $\alpha = 0.05$ (p value)	ACC (%)
DCNN1	(16, 16)	(3×3)	(4×4)	(32, 1)	32	50	0.98	0.19	95
					64	50	0.99	0.45	97
					16	100	0.99	0.41	96
					32	100	0.98	0.16	98
					64	100	0.99	0.5	99
					32	150	<u>0.99</u>	—	99
					64	150	0.98	0.45	99
DCNN2	(32, 64, 128)	(3×3)	(6×6)	(32, 32, 1)	16	50	0.71	0.15	90
					32	50	0.74	0.45	91
					64	50	0.73	0.33	92
					16	100	<u>0.79</u>	—	92
					32	100	0.72	0.23	93
					64	100	0.72	0.33	94
DCNN3	(20, 20)	(2×2)	(2×2)	(3×3)	16	50	0.91	0.26	94
					64	50	<u>0.94</u>	—	97
					32	100	0.92	0.33	98
					64	100	0.91	0.17	99
					32	150	0.90	0.08	99

Conv.- convolutional; f- number of filters per layer; n- number of neurons per layer; FC- fully; connected; u- units; AUC and ACC - mean of AUC and ACC metrics over ten folds; underlined AUC value is the Wilcoxon test pivot value; ACC - mean of accuracy.

of lower-order moments [25]. This optimizer was designed to combine the advantages of the well-known optimizers *AdaGrad* and *RMSProp* [26].

4) *Validation metrics*: the classification performance of the proposed method was based on the AUC and Accuracy (ACC) metrics. The statistical comparison among all classification schemes was conducted using the Wilcoxon test with a significance decision value of 5% ($\alpha = 0.05$) for a two-tailed test [27]. This test ranks the differences in performances of two MLCs [28] and thus, allowed us to select the best classification model.

5) *Selection criteria*: the best model (output) was selected based on the following criteria: (1) the model with the highest AUC score statistically per architecture, (2) if there was a tie rating performance in AUC scores, the one with lower algorithm complexity is preferred, and (3) the highest AUC score statistically among all the selected models by the two previous rules. More than one model per architecture can be selected if there is not a significant AUC based difference among them. This exception is only valid for the intra-architecture analysis. Thus, the proposed method provides only one classification model as output. All implementation was done in Python programming language version 3.7.4 [29].

III. RESULTS AND DISCUSSION

A. Performance evaluation of the proposed method

The DCNN1 architecture provided seven out of nine classification models using the first selection criterion. This set of classifiers did not represent statistical differences in terms of AUC performance when compared to each other. The AUC range of variation was above the 0.95, which is an outstanding classification threshold for any classification problem. Although the higher AUC score of 0.99 was reached by the model using a batch size of 32 units and 150 epochs (iterations), the remaining models performed similarly statistically. According to the second selection criterion, the

selected classification model in this architecture is the one implementing a batch size of 32 units, 50 epochs, and AUC score of 0.98 (see Table I, bold line).

Likewise, DCNN2 architecture was able to produce six out of nine classification models that were similar statistically in AUC performances. The range of AUC variation in this set was between 0.71 and 0.79, which are not good enough scores to tackle the problem at hand. The highest AUC score of 0.79 was reached by the model with 16 units of batch size and 100 epochs. But, the model composed of the same batch size and 50 epochs, which touched an AUC value of 0.71 was selected as the best model from this architecture, taking into consideration the second selection criterion (see Table I, bold line). Similarly, in the DCNN3 architecture, a total of five out of nine classifiers were highlighted as classification models without statistical difference among them. The AUC scores varied from 0.90 to 0.94, which are considered reasonable scores in the context of spectrogram images classifications. The higher AUC value of 0.94 was obtained by the model composed of a batch size of 64 units and 50 epochs. However, there was another model using the same number of epochs as the highest model, batch size of 16, and AUC score of 0.91, which was selected as the best model inside this architecture according to the second selection criterion (see Table I, bold line).

The combined classification models based on deep CNN and RNN architectures were the worst in terms of AUC performances. The three explored architectures provided AUC scores of 0.50 on all classification models, which means very poor schemes generalization. This effect is extremely linked to the number of samples employed during the models training. Despite using the data augmentation technique and the 10-fold cross-validation method on the experimental dataset before feeding the classifiers, they incur in a poor generalization power (see Fig. 4, left plot) and a week learning (Fig. 4, right plot); both causes are symptoms of overfitting. It should

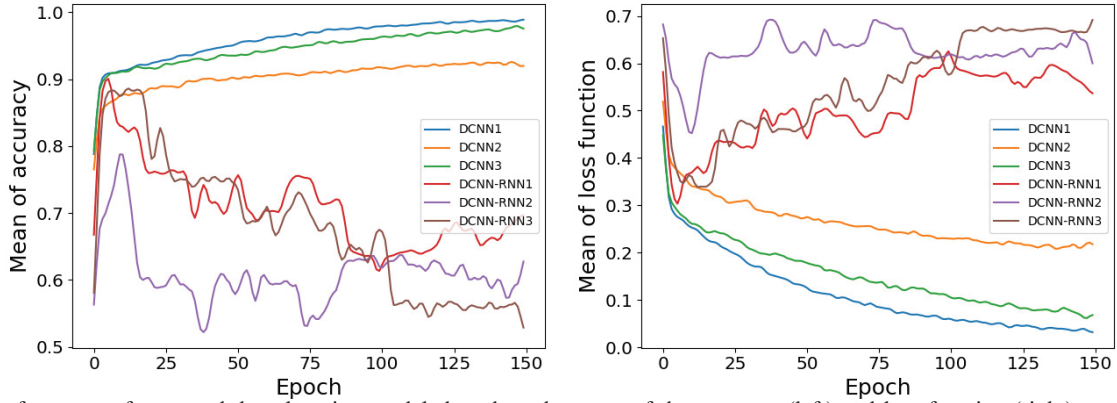


Fig. 4: Performance of proposed deep learning models based on the mean of the accuracy (left) and loss function (right) over ten folds.

be noted that the mean of the loss function never meets the established learning rate on these models, suggesting the inclusion of more samples in the training process.

According to the first two selection criteria, only the deep CNN based classification models provided evidence of successful performance without incurring on overfitting. They performed over the 90% of the mean of ACC in the validation and the loss values converged to the learning rate across the defined epochs (see Fig. 4, left and right plot). Despite the good classification performances, the best selection model in the DCNN2 based architecture reached AUC and ACC scores of 0.71 and 90%, respectively. These values are statistically lower ($p < 0.05$) when compared to the best model selection inside the DCNN3 architecture, which touched AUC and ACC scores of 0.91 and 94%, respectively. The difference in performance is linked to the model complexity inherited from its architecture and the number of samples to train it. The DCNN2 architecture is the most complex among all the developed deep CNN architectures. Thus, it is very reasonable to interpret that this model needs more samples and epochs to learn the feature space properly (see Fig. 4, right plot).

Moreover, the selected classification model using the DCNN1 architecture provided the best performances on both validation metrics. It obtained scores of 0.98 and 95% for the AUC and ACC metrics, respectively. It statistically ($p < 0.05$) overcomes the performance of the remaining models (see Table I). This success is related to the DCNN1 architecture, which employed two convolutional layers with only 16 neurons (filters) per layer (lower than the DCNN3 architecture). Thus, it was able to learn from the provided features space satisfactorily (see Fig. 4, right plot). Regarding the third selection criterion, the selected classification model of the DCNN1 architecture constituted the proposed method output and the most appropriate classifier to face the problem of volcano spectrogram image classification.

B. State of the art based comparison

Although it is not possible to make a direct statistical comparison against some previously developed state of the art methods such as those developed in [3], [10], [15], [30], because they used different experimental conditions, we

TABLE II: Comparison based on the ACC between related previous works and the best selected model produced in this work.

Method	Number of samples	Computed features	Spectrogram images	ACC* (%)
ANN [5]	914	6	no	97
DT [5]	914	3	no	96
ANN [7]	637	17	no	95
RF [7]	637	17	no	93
linear SVM [31]	914	5	no	97
ANN [10]	1033	8	no	94
HMM [3]	512	39	no	90
GMM [8]	667	2	no	94
CNN [15]	15895		yes	97
SVM [30]	105000	102	yes	92
DCNN1 model	1108		yes	95

ACC - accuracy; *values rounded to the closest integer

aimed to carry out the comparison based on the ACC scores reported by them, as it is shown in Table II. The majority of presented machine learning models reached ACC scores ranging from 90 to 97%, being the linear SVM and ANN the models which provided the higher classification performance. The proposed method has similar and superior performance compared to several states of the art methods in terms of ACC scores. That was possible because deep learning-based approaches are able to learn data abstraction from layer to layer, using different mathematical functions. Meanwhile, machine learning methods, except for nonlinear models like ANN, attempt to fit the data with a single mathematical function, which limited the learning ability.

On the other hand, the method developed in [15], used a deep CNN model that achieved an ACC score of 97%. This result was superior when compared to the 95% obtained by the proposed method. However, they classified four types of seismic events instead of two, like in this work. Also, they made the training-test validation using an extensive dataset, which provided a decent number of samples during the model learning.

IV. CONCLUSIONS AND FUTURE WORK

We explored the use of six different deep learning architectures based on deep CNN (DCNN1, DCNN2, and DCNN3) and RNN (DCNN-RNN1, DCNN-RNN2, and DCNN-RNN3) models to classify LP and VT seismic events on a dataset of seismic spectrogram images. The models based

on the combination of deep CNN and RNN architectures reached the worst classification performances. The data augmentation operation helped to reinforce the learning of the DCNN1, DCNN2, and DCNN3 models. But it was not enough for the deep RNN based models, leading then to the overfitting anyway. The DCNN1 was the best model when compared with the other deep CNN based models, attaining AUC and ACC scores of 0.98 and 95%, respectively. Although these values were not the highest values per metric, they did not represent statistical differences against other results that were obtained by more algorithmically complex models. Furthermore, the proposed DCNN1 model showed similar or superior performance when compared to the majority of the state of the art methods in terms of the ACC metric. Therefore it can be considered as a successful scheme to classify LP and VT seismic events based on their spectrogram images.

As future work, we plan to increase the number of samples per class to experiment with more complex architectures like the deep CNN+RNN models and improve the hyperparameter configurations to explore the limits of the implemented models.

ACKNOWLEDGMENT

We thank the Applied Signal Processing and Machine Learning Research Group of UFSQ for providing the computing infrastructure (NVIDIA DGX workstation) to implement and execute the developed source code. The publication of this article was funded by the Academic Articles Publication Fund of Universidad San Francisco de Quito USFQ.

REFERENCES

- [1] R. I. Tilling, "Hazards and climatic impact of subduction-zone volcanism: A Global And Historical Perspective," *Washington DC American Geophysical Union Geophysical Monograph Series*, vol. 96, pp. 331–335, 1996.
- [2] H.-U. Schmincke, "Volcanic hazards, volcanic catastrophes, and disaster mitigation," in *Volcanism*. Springer, 2004, pp. 229–258.
- [3] M. C. Benítez, J. Ramirez, J. C. Segura, J. M. Ibanez, J. Almendros, A. Garcia-Yeguas, and G. Cortes, "Continuous hmm-based seismic-event classification at deception island, antarctica," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 45, no. 1, pp. 138–146, Jan 2007.
- [4] P. Venegas, N. Pérez, D. S. Benítez, R. Lara-Cueva, and M. Ruiz, "Building machine learning models for long-period and volcano-tectonic event classification," in *2019 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON)*. IEEE, 2019, pp. 1–6.
- [5] R. Lara-Cueva, E. V. Carrera, J. F. Morejon, and D. Benítez, "Comparative analysis of automated classifiers applied to volcano event identification," in *2016 IEEE Colombian Conference on Communications and Computing (COLCOM)*, April 2016, pp. 1–6.
- [6] M. Rodgers, P. Smith, D. Pyle, and T. Mather, "Waveform classification and statistical analysis of seismic precursors to the July 2008 vulcanian eruption of soufrière hills volcano, montserrat," in *EGU General Assembly Conference Abstracts*, vol. 18, 2016.
- [7] N. Pérez, P. Venegas, D. Benítez, R. Lara-Cueva, and M. Ruiz, "A new volcanic seismic signal descriptor and its application to a data set from the cotopaxi volcano," *IEEE Transactions on Geoscience and Remote Sensing*, pp. 1–11, 2020.
- [8] P. Venegas, N. Pérez, D. Benítez, R. Lara-Cueva, and M. Ruiz, "Combining filter-based feature selection methods and gaussian mixture model for the classification of seismic events from cotopaxi volcano," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 6, pp. 1991–2003, 2019.
- [9] M. Curilem, J. Vergara, C. San Martin, G. Fuentealba, C. Cardona, F. Huenupan, M. Chacón, M. S. Khan, W. Hussein, and N. B. Yoma, "Pattern recognition applied to seismic signals of the llaima volcano (chile): An analysis of the events' features," *Journal of Volcanology and Geothermal Research*, vol. 282, pp. 134–147, 2014.
- [10] G. Curilem, J. Vergara, G. Fuentealba, G. Acuña, and M. Chacón, "Classification of seismic signals at villarrica volcano (chile) using neural networks and genetic algorithms," *Journal of volcanology and geothermal research*, vol. 180, no. 1, pp. 1–8, 2009.
- [11] A. Bueno, C. Benítez, S. De Angelis, A. Díaz Moreno, and J. M. Ibáñez, "Volcano-seismic transfer learning and uncertainty quantification with bayesian neural networks," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 58, no. 2, pp. 892–902, 2020.
- [12] N. Pérez, D. Benítez, F. Grijalva, R. Lara-Cueva, M. Ruiz, and J. Aguilar, "Eseismic: Towards an ecuadorian volcano seismic repository," *Journal of Volcanology and Geothermal Research*, p. 106855, 2020.
- [13] H.-C. Shin, H. R. Roth, M. Gao, L. Lu, Z. Xu, I. Noguees, J. Yao, D. Mollura, and R. M. Summers, "Deep convolutional neural networks for computer-aided detection: Cnn architectures, dataset characteristics and transfer learning," *IEEE transactions on medical imaging*, vol. 35, no. 5, pp. 1285–1298, 2016.
- [14] M. Titos, A. Bueno, L. García, and C. Benítez, "A deep neural networks approach to automatic recognition systems for volcano-seismic events," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 11, no. 5, pp. 1533–1544, 2018.
- [15] M. Curilem, J. P. Canário, L. Franco, and R. A. Rios, "Using cnn to classify spectrograms of seismic events from llaima volcano (chile)," in *2018 International Joint Conference on Neural Networks (IJCNN)*, July 2018, pp. 1–8.
- [16] H. Lecun, Bengio, "Deep learning," *Nature*, vol. 521, pp. 436 – 444, 05 2015.
- [17] J. Wang, Y. Yang, J. Mao, Z. Huang, C. Huang, and W. Xu, "CNN-RNN: A Unified Framework for Multi-label Image Classification," *arXiv e-prints*, p. arXiv:1604.04573, Apr. 2016.
- [18] A. Canziani, A. Paszke, and E. Culurciello, "An Analysis of Deep Neural Network Models for Practical Applications," *arXiv e-prints*, p. arXiv:1605.07678, May 2016.
- [19] Q. Yu, Y. Yang, Y.-Z. Song, T. Xiang, and T. Hospedales, "Sketch-a-Net that Beats Humans," *arXiv e-prints*, p. arXiv:1501.07873, Jan. 2015.
- [20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [21] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 7, pp. 3639–3655, 2017.
- [22] Y. K. Jain and S. K. Bhandare, "Min max normalization based data perturbation method for privacy protection," *International Journal of Computer & Communication Technology*, vol. 2, no. 8, pp. 45–50, 2011.
- [23] A. Mikołajczyk and M. Grochowski, "Data augmentation for improving deep learning in image classification problem," in *2018 International Interdisciplinary PhD Workshop (IIPhDW)*, 2018, pp. 117–122.
- [24] F. G. López, M. G. Torres, B. M. Batista, J. A. M. Pérez, and J. M. Moreno-Vega, "Solving feature subset selection problem by a parallel scatter search," *European Journal of Operational Research*, vol. 169, no. 2, pp. 477–489, 2006.
- [25] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv e-prints*, p. arXiv:1412.6980, Dec. 2014.
- [26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [27] M. Hollander, D. A. Wolfe, and E. Chicken, *Nonparametric statistical methods*. John Wiley & Sons, 2013, vol. 751.
- [28] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine learning research*, vol. 7, no. Jan, pp. 1–30, 2006.
- [29] Python Core Team, *Python 3.6.9: A dynamic, open source programming language*. Python Software Foundation, 2019. [Online]. Available: <https://www.python.org/>.
- [30] M. Malfante, M. Dalla Mura, J.-P. Métaxian, J. I. Mars, O. Macedo, and A. Inza, "Machine learning for volcano-seismic signals: Challenges and perspectives," *IEEE Signal Processing Magazine*, vol. 35, no. 2, pp. 20–30, 2018.
- [31] R. A. Lara-Cueva, D. S. Benítez, E. V. Carrera, M. Ruiz, and J. L. Rojo-Álvarez, "Automatic recognition of long period events from volcano tectonic earthquakes at cotopaxi volcano," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 54, no. 9, pp. 5247–5257, Sept 2016.